

[프롬프트]

안녕 챗지피티! 오늘은 파이썬프로그래밍입문 3차 과제를 하는 날이야. 본 파일 3페이지를 보면 3차 과제에서는 무한반복, 반복문을 이용하여 추가 설계를 해야한다는 것을 알 수 있어.

1. 시나리오 제목

[자취 라이프 매니저] 냉장고 관리 시스템

2. 시나리오 (5~10줄)

이 프로그램은 매달 식비 부족에 시달리는 자취생들을 위해 냉장고 속 식재료와 지출을 스마트하게 관리해 주는 '자취 라이프 매니저' 시스템입니다.

가장 먼저, 이번 달의 소중한 식비 예산을 설정하고 오늘 새로 산 식재료의 이름과 가격, 그리고 남은 유효기간(혹은 무게)을 기록하는 것으로 관리를 시작합니다. 그다음에는 입력한 정보를 바탕으로 이 재료가 내 전체 예산에서 어느 정도의 비중을 차지하는지 '식비 점유율'을 계산해 봅니다. 이렇게 계산된 점유율이 너무 높으면 '지출 주의' 경고를, 적절하면 '합리적 소비'라는 판정을 내려줍니다. 만약 가격도 저렴한데 영양가까지 높은 '갓성비' 재료를 샀다면 '알뜰 자취왕'이라는 특별한 칭호를 부여하여 동기부여를 해줄 것입니다. 나중에는 단순히 하나씩 기록하는 것이 아니라, 냉장고 속의 수많은 재료를 커다란 바구니(리스트)에 담아 유통기한 순으로 정렬하거나 전체 남은 예산을 실시간으로 조회할 수 있게 만들 것입니다. 최종적으로는 사용자가 메뉴판에서 번호를 눌러 기능을 선택하고, '자취 종료(로그아웃)'를 하기 전까지는 계속해서 식재료를 관리할 수 있는 완벽한 자취 보조 소프트웨어로 완성될 계획입니다.

3. 예상 기능 및 메뉴 (최소 5개)

(예시)

신규 식재료 등록 (이름, 가격, 유통기한 등)

식비 점유율 및 잔액 계

지출 등급 판정 (알뜰/주의/과소비)

'알뜰 자취왕' 비밀 칭호 부여

냉장고 식재료 명단 전체 출력

프로그램 종료



[버전별 개발 일지 & AI 협업 기록]



[1차 과제: V1.0] 시나리오 기획



AI 파트너십 과정

(예시)

내용 1 : 자취생 맞춤형 컨셉 도출

프롬프트 요약: ""자취생인 내가 실용적으로 사용할 수 있으면서도 재미있는 파이썬 프로젝트 주제를 추천해줘.""

적용 내용: AI가 제안한 여러 아이디어 중 식재료 관리와 식비 계산을 결합한 '자취 라이프 매니저' 컨셉을 채택함. 대학생의 실생활과 밀접한 주제라 개발 동기가 확실해짐.

내용 2 : 데이터 구조 및 로드맵 설

프롬프트 요약: "1차 과제 요건(변수 5개, 자료형 3개 이상)을 충족하면서 자취생에게 꼭 필요한 변수 구성과 확장 계획을 세워줘."

적용 내용: 재료명(문자열), 예산/가격(정수형), 무게/점유율(실수형) 등 자료형을 다양하게 배치하여 과제 요건을 충족함. 이후 단계에서 리스트와 함수를 통해 '유통기한 관리'로 확장하기 위한 기초 설계를 마침.

📁 증빙 자료:

[1차_AI협업캡처.pdf 첨부 완료] (첨부 후 링크)

https://github.com/MJU-PY-2026-1/mju-py-homework-yubiny205-crypto-1/blob/main/1%EC%B0%A8%EA%B3%BC%EC%A0%9C_AI%ED%98%91%EC%97%85%EC%A6%9D%EB%A%85.pdf

🟩 [2차 과제: V1.0] 입출력 + 리스트 + 조건문 - 향후 작성 예정

🌟 2차 과제 내용:

이번 과제는 [자취 라이프 매니저] 냉장고 관리 시스템 V1.0 구현으로, 사용자로부터 식비 예산과 식재료 정보를 입력받아 리스트에 저장하고, 지출 금액과 예산 점유율을 계산하는 프로그램을 만드는 것입니다.

input()으로 사용자 이름, 이번 달 식비 예산, 식재료 이름, 가격, 남은 유통기한, 영양 점수를 입력받아 리스트에 저장했습니다.

for문을 사용하여 최소 3개의 식재료 데이터를 입력받고, append()를 활용해 식재료 이름과 가격, 유통기한, 영양 점수를 각각 리스트에 추가했습니다.

sum(), len(), max(), sort(), insert() 등을 활용하여 리스트 데이터를 조작하고, 총 지출 금액과 남은 예산, 식비 점유율을 계산했습니다.

if-elif-else 조건문을 사용하여 지출 등급을 알뜰 소비 / 적정 소비 / 과소비 주의로 판정했습니다.

and, or, not 논리 연산자와 중첩 조건문을 활용하여 예산과 영양 점수를 함께 고려한 알뜰 자취왕 특별 칭호를 부여했습니다.

🤖 AI 파트너십 과정

문제 1: 변수 이름 오타로 인해 없는 항목이라고 뜨는 문제

원인: 리스트나 변수 이름을 선언할 때 사용한 이름과 출력 또는 계산할 때 사용한 이름이 서로 달라서 오류가 발생했습니다. 예를 들어 food_prices로 만든 리스트를 다른 줄에서 food_price처럼 잘못 입력하면 파이썬이 해당 변수를 찾지 못했습니다.

해결: 변수 이름을 한 줄씩 확인하면서 선언한 이름과 사용하는 이름을 모두 동일하게 수정했습니다. 이번 경험을 통해 변수 이름은 처음부터 의미 있게 정하고, 같은 이름을 끝까지 일관되게 사용하는 것이 중요하다는 것을 알게 되었습니다.

문제 2: 필요한 코드 한 줄을 빠뜨려 실행 결과가 원하는 대로 나오지 않은 문제

원인: 식재료 정보를 입력받은 뒤 리스트에 추가하는 append() 줄이나, 총 지출 금액을 누적하는 total_price += price 줄을 빠뜨려 계산 결과가 제대로 반영되지 않았습니다.

해결: 입력 → 리스트 저장 → 계산 → 출력 순서로 코드 흐름을 다시 확인하며 빠진 줄을 추가했습니다. 특히 반복문 안에서 입력받은 값이 리스트에 저장되고 있는지, 총 지출 금액에 누적되고 있는지 확인했습니다.

문제 3: input으로 입력받은 숫자가 계산에 바로 사용되지 않는 문제

원인: input()으로 입력받은 값은 기본적으로 문자열이기 때문에 가격이냐 예산 계산에 바로 사용할 수 없었습니다.

해결: 예산과 가격은 int(), 영양 점수는 float()로 형변환하여 계산이 가능하도록 수정했습니다.

문제 4: 조건문 결과가 예상과 다르게 출력되는 문제

원인: if, elif, else의 순서를 잘못 배치하거나 조건 범위를 애매하게 작성하여 원하는 등급이 출력되지 않았습니다.

해결: 식비 점유율이 큰 경우부터 차례대로 검사하도록 조건문의 순서를 정리했습니다.

또한 and, or, not 연산자를 사용할 때 각각의 조건이 어떤 의미인지 확인하며 수정했습니다.

📁 증빙 자료:

[2차_AI협업캡처.pdf 첨부 완료] (첨부 후 링크)

[2차과제_실행결과.jpg]

아래는 이때까지 내가 1차, 2차 과제를 하면서 작성한 계획 및 보고서의 내용이고

이 밑에는 2차 과제까지 완료한 후의 코드를 보여줄게

print('자취 라이프 매니저 - 냉장고 관리 시스템')

```
name = input('사용자 이름:')
monthly_budget = int(input('이번달 식비 예산:'))
warning_rate = 50.0
total_price = 0
```

```
food_names = []
food_prices = []
food_dates = []
nutrition_scores = []
```

```
print('\n식재료 3개를 등록합니다.')
```

```
for i in range(3):
    print(f'\n{i+1}번째 식재료 입력')
    food_name = input('식재료 이름:')
    price = int(input('가격:'))
    date = int(input('남은 유통기한(일):'))
    nutrition = float(input('영양 점수(0~10): '))
```

```
    food_names.append(food_name)
    food_prices.append(price)
    food_dates.append(date)
    nutrition_scores.append(nutrition)
```

```
    total_price += price
```

```
food_names.insert(0, '냉장고 목록')
food_prices.sort()
```

```
food_count = len(food_dates)
max_price = max(food_prices)
total_nutrition = sum(nutrition_scores)
average_nutrition = total_nutrition / 3
```

```
remaining_budget = monthly_budget - total_price
budget_rate = (total_price / monthly_budget) * 100
```

```

print(f'등록한 식재료 개수: {food_count}개')
print(f'총 지출 금액: {total_price}원')
print(f'남은 예산: {remaining_budget}원')
print(f'식비 점유율: {budget_rate: .2f}%')

if budget_rate >= warning_rate:
    print('지출 등급: 과소비 주의')
elif budget_rate >= 30:
    print('지출 등급: 걱정 소비')
else:
    print('지출 등급: 알뜰 소비')

if remaining_budget > 0:
    if budget_rate < 30 and average_nutrition >= 7:
        print('특별 칭호: 알뜰 자취왕')
    elif budget_rate < 50 or average_nutrition >= 6:
        print('특별 칭호: 절약 연습생')
else:
    print('예산을 초과했습니다.')
    if not remaining_budget > 0:
        print('다음 장보기에서는 가격을 조금 더 신경 써야 합니다.')

print('\n[냉장고 식재료 이름 목록]')
print(food_names)

print('\n[가격 오름차순 목록]')
print(food_prices)

print('\n프로그램을 종료합니다.')

```

이때까지는 이렇게 작성하였어 3차 과제를 어떤 식으로 진행해보면 좋을까?

[답변]

안녕 유빈! 이번 3차 과제는 지금 코드에서 “한 번 실행하고 끝나는 프로그램”을 “메뉴를 누르면서 계속 관리하는 프로그램”으로 바꾸는 단계라고 보면 돼. 파일 3페이지에서도 3차 과제 핵심이 **while True / break**를 이용한 무한 반복, **for**문을 이용한 리스트 순회, 메뉴 선택에 따른 기능 실행으로 정리되어 있어.

지금 2차 코드가 이미 리스트, 조건문, 계산 기능까지 잘 들어가 있어서 3차는 완전 갈아엎기보다는 메뉴판을 만들고, 기존 기능들을 메뉴 안으로 넣는 방식이 제일 자연스러워.

3차 과제 방향

이번 버전은 이렇게 잡으면 좋아.

[자취 라이프 매니저 V2.0] 메뉴형 냉장고 관리 시스템

기존에는 식재료 3개를 한 번에 입력하고 결과를 출력한 뒤 종료했지만, 3차에서는 사용자가 직접 메뉴 번호를 선택하여 원하는 기능을 반복 실행할 수 있도록 확장한다. `while True`를 사용하여 프로그램이 계속 실행되도록 만들고, 사용자가 종료 메뉴를 선택하면 `break`로 반복을 끝낸다. 또한 `for`문을 이용해 냉장고 식재료 목록을 하나씩 출력하고, 각 식재료의 가격, 유통기한, 영양 점수를 함께 보여준다.

추천 메뉴 구성

3차 과제에서는 최소한 아래 정도면 딱 좋아.

[자취 라이프 매니저 메뉴]

1. 신규 식재료 등록
2. 냉장고 식재료 전체 출력
3. 식비 점유율 및 남은 예산 확인
4. 지출 등급 판정
5. 알뜰 자취왕 칭호 확인
6. 유통기한 임박 식재료 확인
0. 프로그램 종료

여기서 3차 과제 핵심을 잘 보여주는 메뉴는 특히 이거야.

1번은 `while True` 안에서 계속 식재료를 추가할 수 있다는 점을 보여줌.

2번은 `for`문으로 리스트를 순회하는 기능.

0번은 `break` 사용.

6번은 조건문 + 반복문을 같이 쓰는 기능이라 점수 잘 챙기기 좋음.

기존 코드에서 바뀌는 핵심 구조

지금은 이 구조잖아.

식재료 3개 입력
계산
출력
종료

3차에서는 이렇게 바꾸면 돼.

초기 설정 입력

```

while True:
    메뉴 출력
    메뉴 번호 입력

    if 메뉴 == 1:
        식재료 등록
    elif 메뉴 == 2:
        전체 목록 출력
    elif 메뉴 == 3:
        예산 계산 출력
    elif 메뉴 == 4:
        지출 등급 판정
    elif 메뉴 == 5:
        특별 칭호 확인
    elif 메뉴 == 6:
        유통기한 임박 식재료 출력
    elif 메뉴 == 0:
        종료
        break
    else:
        잘못된 입력 안내

```

이게 3차 과제 의도랑 제일 잘 맞아.

3차 코드 예시

지금 네 코드 스타일을 유지해서 너무 어렵지 않게 만들면 이렇게 가면 돼.

```

print('자취 라이프 매니저 - 냉장고 관리 시스템 V2.0')

name = input('사용자 이름:')
monthly_budget = int(input('이번달 식비 예산:'))

warning_rate = 50.0

food_names = []
food_prices = []
food_dates = []
nutrition_scores = []

while True:
    print('\n===== 자취 라이프 매니저 메뉴 =====')
    print('1. 신규 식재료 등록')
    print('2. 냉장고 식재료 전체 출력')
    print('3. 식비 점유율 및 남은 예산 확인')

```

```
print('4. 지출 등급 판정')
print('5. 알뜰 자취왕 칭호 확인')
print('6. 유통기한 임박 식재료 확인')
print('0. 프로그램 종료')
print('=====')
```

```
menu = input('메뉴 번호를 선택하세요: ')
```

```
if menu == '1':
```

```
    print('\n[신규 식재료 등록]')
    food_name = input('식재료 이름:')
    price = int(input('가격:'))
    date = int(input('남은 유통기한(일):'))
    nutrition = float(input('영양 점수(0~10): '))
```

```
    food_names.append(food_name)
    food_prices.append(price)
    food_dates.append(date)
    nutrition_scores.append(nutrition)
```

```
    print(f'{food_name} 등록이 완료되었습니다.')
```

```
elif menu == '2':
```

```
    print('\n[냉장고 식재료 전체 목록]')
```

```
    if len(food_names) == 0:
```

```
        print('아직 등록된 식재료가 없습니다.')
```

```
    else:
```

```
        for i in range(len(food_names)):
```

```
            print(f'{i+1}. {food_names[i]} / 가격: {food_prices[i]}원 / 유통기한: {food_dates[i]}일  
/ 영양 점수: {nutrition_scores[i]}점')
```

```
elif menu == '3':
```

```
    print('\n[식비 점유율 및 남은 예산 확인]')
```

```
    total_price = sum(food_prices)
```

```
    remaining_budget = monthly_budget - total_price
```

```
    if monthly_budget > 0:
```

```
        budget_rate = (total_price / monthly_budget) * 100
```

```
    else:
```

```
        budget_rate = 0
```

```
    print(f'총 지출 금액: {total_price}원')
```

```
    print(f'남은 예산: {remaining_budget}원')
```

```
    print(f'식비 점유율: {budget_rate:.2f}%')
```

```
elif menu == '4':
```

```

print('\n[지출 등급 판정]')

total_price = sum(food_prices)

if monthly_budget > 0:
    budget_rate = (total_price / monthly_budget) * 100
else:
    budget_rate = 0

if budget_rate >= warning_rate:
    print('지출 등급: 과소비 주의')
elif budget_rate >= 30:
    print('지출 등급: 적정 소비')
else:
    print('지출 등급: 알뜰 소비')

elif menu == '5':
    print('\n[특별 칭호 확인]')

    total_price = sum(food_prices)
    remaining_budget = monthly_budget - total_price

    if len(nutrition_scores) > 0:
        average_nutrition = sum(nutrition_scores) / len(nutrition_scores)
    else:
        average_nutrition = 0

    if monthly_budget > 0:
        budget_rate = (total_price / monthly_budget) * 100
    else:
        budget_rate = 0

    if remaining_budget > 0:
        if budget_rate < 30 and average_nutrition >= 7:
            print('특별 칭호: 알뜰 자취왕')
        elif budget_rate < 50 or average_nutrition >= 6:
            print('특별 칭호: 절약 연습생')
        else:
            print('특별 칭호: 자취 입문자')
    else:
        print('예산을 초과했습니다.')
        print('다음 장보기에서는 가격을 조금 더 신경 써야 합니다.')

elif menu == '6':
    print('\n[유통기한 임박 식재료 확인]')

    if len(food_names) == 0:
        print('아직 등록된 식재료가 없습니다.')

```

```

else:
    found = False

    for i in range(len(food_names)):
        if food_dates[i] <= 3:
            print(f'{food_names[i]}: 유통기한이 {food_dates[i]}일 남았습니다.')
            found = True

    if not found:
        print('유통기한이 임박한 식재료가 없습니다.')

elif menu == '0':
    print('\n자취 라이프 매니저를 종료합니다.')
    print(f'{name}님의 냉장고 관리를 마칩니다.')
    break

else:
    print('잘못된 메뉴 번호입니다. 다시 선택해주세요.')

```

보고서에는 이렇게 쓰면 좋음

3차 과제 내용은 이런 식으로 정리하면 자연스러워.

🟡 [3차 과제: V2.0] 반복문 + 메뉴 시스템

✨ 3차 과제 내용:

이번 과제는 [자취 라이프 매니저] 냉장고 관리 시스템을 메뉴형 프로그램으로 확장하는 것입니다. 기존 2차 과제에서는 식재료 3개를 한 번 입력하고 결과를 출력한 뒤 프로그램이 종료되었지만, 3차 과제에서는 **while True**를 사용하여 사용자가 원하는 기능을 계속 선택할 수 있도록 수정했습니다.

사용자는 메뉴에서 신규 식재료 등록, 전체 목록 출력, 식비 점유율 확인, 지출 등급 판정, 특별 칭호 확인, 유통기한 임박 식재료 확인, 프로그램 종료 기능을 선택할 수 있습니다. 0번 종료 메뉴를 선택하면 **break**를 사용하여 반복문을 빠져나오도록 구현했습니다.

또한 **for**문을 이용하여 **food_names, food_prices, food_dates, nutrition_scores** 리스트에 저장된 식재료 정보를 순서대로 출력했습니다. 유통기한이 3일 이하로 남은 식재료를 찾아 알려주는 기능도 추가하여 단순 계산 프로그램에서 실제 냉장고 관리 프로그램에 더 가까워지도록 개선했습니다.

AI 협업 기록은 이렇게 쓰면 됨

🤖 AI 파트너십 과정

문제 1: 프로그램이 한 번 실행되고 바로 종료되는 문제

원인: 2차 과제 코드에서는 식재료 3개를 입력받은 뒤 결과를 출력하고 프로그램이 끝나는 구조였습니다. 따라서 사용자가 추가로 식재료를 등록하거나 다시 계산 결과를 확인할 수 없었습니다.

해결: AI와 상의하여 **while True**를 사용한 메뉴 반복 구조로 코드를 변경했습니다. 사용자가 0번 종료 메뉴를 선택하기 전까지 프로그램이 계속 실행되도록 만들었고, 종료 시에는 **break**를 사용하여 반복문을 빠져나오게 했습니다.

문제 2: 리스트에 저장된 식재료 정보를 보기 어렵던 문제

원인: 기존 코드에서는 식재료 이름 리스트와 가격 리스트를 따로 출력하여 각 식재료의 가격, 유통기한, 영양 점수를 한눈에 확인하기 어려웠습니다.

해결: **for**문과 **range(len(food_names))**를 사용하여 같은 인덱스에 저장된 식재료 이름, 가격, 유통기한, 영양 점수를 함께 출력하도록 수정했습니다. 이를 통해 냉장고 목록을 더 보기 쉽게 확인할 수 있었습니다.

문제 3: 등록된 식재료가 없을 때 계산이나 출력이 어색한 문제

원인: 사용자가 아무 식재료도 등록하지 않은 상태에서 전체 목록 출력이나 특별 칭호 확인 메뉴를 선택하면 출력할 데이터가 없거나 평균 영양 점수 계산이 어려울 수 있었습니다.

해결: **len(food_names) == 0** 조건을 사용하여 등록된 식재료가 없는 경우 안내 문구를 출력하도록 했습니다. 또한 영양 점수 평균을 계산할 때 리스트 길이가 0인지 먼저 확인하여 오류가 발생하지 않도록 했습니다.

개인적으로는 3차에서 “유통기한 임박 식재료 확인” 기능 넣는 게 아주 좋아 보여. 교수님 입장에서 봤을 때도 그냥 메뉴만 만든 게 아니라, 냉장고 관리 시스템이라는 주제랑 반복문 기능이 딱 맞게 연결돼서 과제 의도가 잘 살아나.

[프롬프트]

내가 3차 과제에 대해 더 상세한 설명이 담긴 파일을 너에게 추가로 보내줄게

[답변]

좋아, 이 파일 보니까 내가 아까 말한 방향에서 중요한 조건이 몇 개 더 추가돼 있어. 특히 3차는 단순히 **while True** 메뉴만 넣으면 끝이 아니라, 함수(**def**), 매개변수, **return**, **global**까지 반드시 들어가야 해.

파일 기준으로 3차 과제 필수 조건은 이거야. 4페이지에 따르면 V2.0 코드는 메뉴/함수/전역변수 활용이 핵심이고, **while True** 무한루프, **break**, 최소 3개 이상의

함수 분리, 매개변수 1개 이상, **return** 1개 이상, **global** 1회 이상 사용이 필수야.
평가표에서도 이 조건들이 코드 및 실행 점수 3점짜리 핵심 조건으로 들어가 있어.

그래서 3차 과제는 이렇게 진행하면 제일 안전해.

3차 과제 방향 수정본

[자취 라이프 매니저 **V2.0**] 함수형 메뉴 냉장고 관리 시스템

기존 2차 과제에서는 식재료 3개를 입력받고 결과를 한 번 출력한 뒤 종료했지만, 3차 과제에서는 메뉴를 통해 사용자가 원하는 기능을 반복 실행할 수 있도록 확장한다.
while True를 사용하여 메인 메뉴가 계속 출력되게 하고, 사용자가 종료 메뉴를 선택했을 때만 **break**로 프로그램을 종료한다.
또한 기존의 입력, 출력, 계산, 판정 기능을 각각 함수로 분리하여 메인 로직을 간단하게 구성한다.

넣으면 좋은 함수 구성

3차 과제 조건을 만족하려면 최소 3개 함수가 필요하니까, 이 정도가 딱 좋아.

show_menu()

메뉴 출력 함수.
매개변수, **return** 없어도 됨.

add_food()

신규 식재료 등록 함수.
여기서 **global total_price**를 사용하면 **global** 조건을 자연스럽게 만족할 수 있음.

show_food_list()

냉장고 전체 목록 출력 함수.
for문으로 리스트 순회.

calculate_budget_rate(total_price, monthly_budget)

식비 점유율 계산 함수.
이 함수가 매개변수 조건과 **return** 조건을 동시에 만족함.

`show_budget_status()`

총 지출, 남은 예산, 식비 점유율 출력 함수.

`judge_spending()`

지출 등급 판정 함수.

`check_expiring_food()`

유통기한 임박 식재료 확인 함수.

메뉴 구성 추천

1. 신규 식재료 등록
2. 냉장고 식재료 전체 출력
3. 식비 점유율 및 남은 예산 확인
4. 지출 등급 판정
5. 알뜰 자취왕 칭호 확인
6. 유통기한 임박 식재료 확인
0. 프로그램 종료

이 메뉴면 과제 조건과 네 시나리오가 잘 맞아. 특히 파일 2페이지에 제출 항목으로 **V2.0** 코드 작성, **main.py** 수정, 실행결과 캡처, **AI협업증명3.pdf**가 필요하다고 되어 있으니, 실행할 때는 메뉴가 보이고 기능이 작동하는 화면을 캡처해두면 돼.

3차 코드 뼈대는 이렇게 가면 좋음

아까 코드보다 이번 조건에 맞게 함수형 구조로 바꾸는 게 맞아.

```
print('자취 라이프 매니저 - 냉장고 관리 시스템 V2.0')
```

```
name = input('사용자 이름: ')
monthly_budget = int(input('이번달 식비 예산: '))
```

```
warning_rate = 50.0
total_price = 0
```

```
food_names = []
food_prices = []
food_dates = []
```

```
nutrition_scores = []
```

```
def show_menu():
```

```
    print('\n===== 자취 라이프 매니저 메뉴 =====')
    print('1. 신규 식재료 등록')
    print('2. 냉장고 식재료 전체 출력')
    print('3. 식비 점유율 및 남은 예산 확인')
    print('4. 지출 등급 판정')
    print('5. 알뜰 자취왕 칭호 확인')
    print('6. 유통기한 임박 식재료 확인')
    print('0. 프로그램 종료')
    print('=====')
```

```
def add_food():
```

```
    global total_price

    print('\n[신규 식재료 등록]')
    food_name = input('식재료 이름: ')
    price = int(input('가격: '))
    date = int(input('남은 유통기한(일): '))
    nutrition = float(input('영양 점수(0~10): '))

    food_names.append(food_name)
    food_prices.append(price)
    food_dates.append(date)
    nutrition_scores.append(nutrition)

    total_price += price

    print(f'{food_name} 등록이 완료되었습니다.')
```

```
def show_food_list():
```

```
    print('\n[냉장고 식재료 전체 목록]')

    if len(food_names) == 0:
        print('아직 등록된 식재료가 없습니다.')
    else:
        for i in range(len(food_names)):
            print(f'{i+1}. {food_names[i]} / 가격: {food_prices[i]}원 / 유통기한: {food_dates[i]}일 /  
영양 점수: {nutrition_scores[i]}점')
```

```
def calculate_budget_rate(total, budget):
```

```
    if budget > 0:
        rate = (total / budget) * 100
```

```
else:  
    rate = 0
```

```
return rate
```

```
def show_budget_status():  
    remaining_budget = monthly_budget - total_price  
    budget_rate = calculate_budget_rate(total_price, monthly_budget)  
  
    print('\n[식비 점유율 및 남은 예산 확인]')  
    print(f'총 지출 금액: {total_price}원')  
    print(f'남은 예산: {remaining_budget}원')  
    print(f'식비 점유율: {budget_rate:.2f}%')
```

```
def judge_spending():  
    budget_rate = calculate_budget_rate(total_price, monthly_budget)  
  
    print('\n[지출 등급 판정]')  
  
    if budget_rate >= warning_rate:  
        print('지출 등급: 과소비 주의')  
    elif budget_rate >= 30:  
        print('지출 등급: 적정 소비')  
    else:  
        print('지출 등급: 알뜰 소비')
```

```
def check_title():  
    remaining_budget = monthly_budget - total_price  
    budget_rate = calculate_budget_rate(total_price, monthly_budget)  
  
    if len(nutrition_scores) > 0:  
        average_nutrition = sum(nutrition_scores) / len(nutrition_scores)  
    else:  
        average_nutrition = 0  
  
    print('\n[특별 칭호 확인]')  
  
    if remaining_budget > 0:  
        if budget_rate < 30 and average_nutrition >= 7:  
            print('특별 칭호: 알뜰 자취왕')  
        elif budget_rate < 50 or average_nutrition >= 6:  
            print('특별 칭호: 절약 연습생')  
        else:  
            print('특별 칭호: 자취 입문자')  
    else:
```

```

    print('예산을 초과했습니다.')
    print('다음 장보기에서는 가격을 조금 더 신경 써야 합니다.')

def check_expiring_food():
    print('\n[유통기한 임박 식재료 확인]')

    if len(food_names) == 0:
        print('아직 등록된 식재료가 없습니다.')
    else:
        found = False

        for i in range(len(food_names)):
            if food_dates[i] <= 3:
                print(f'{food_names[i]}: 유통기한이 {food_dates[i]}일 남았습니다.')
                found = True

        if not found:
            print('유통기한이 임박한 식재료가 없습니다.')

while True:
    show_menu()
    menu = input('메뉴 번호를 선택하세요: ')

    if menu == '1':
        add_food()
    elif menu == '2':
        show_food_list()
    elif menu == '3':
        show_budget_status()
    elif menu == '4':
        judge_spending()
    elif menu == '5':
        check_title()
    elif menu == '6':
        check_expiring_food()
    elif menu == '0':
        print('\n자취 라이프 매니저를 종료합니다.')
        print(f'{name}님의 냉장고 관리를 마칩니다.')
        break
    else:
        print('잘못된 메뉴 번호입니다. 다시 선택해주세요.')

```

이 코드가 과제 조건을 만족하는지 체크

조건

코드에서 만족하는 부분

<code>while True</code> 무한 반복	맨 아래 메인 메뉴 반복
<code>break</code> 종료	<code>menu == '0'</code> 일 때 종료
함수 3개 이상	<code>show_menu</code> , <code>add_food</code> , <code>show_food_list</code> 등 7개
매개변수 1개 이상	<code>calculate_budget_rate(total, budget)</code>
<code>return</code> 1개 이상	<code>return rate</code>
<code>global</code> 1회 이상	<code>add_food()</code> 안의 <code>global total_price</code>
<code>for</code> 문 리스트 순회	<code>show_food_list()</code> , <code>check_expiring_food()</code>
기존 기능 유지	식재료 등록, 예산 계산, 등급 판정, 칭호 부여

이대로면 평가표의 핵심 조건은 거의 다 맞아.

README에는 이렇게 쓰면 좋음

■ [3차 과제: V2.0] 메뉴/함수/반복문 기반 냉장고 관리 시스템

✨ 3차 과제 내용:

이번 과제는 [자취 라이프 매니저] 냉장고 관리 시스템을 V2.0으로 확장하는 것입니다. 기존 2차 과제에서는 식재료 정보를 입력받고 계산 결과를 한 번 출력한 뒤 프로그램이 종료되었지만, 3차 과제에서는 **while True** 반복문을 사용하여 사용자가 원하는 기능을 계속 선택할 수 있는 메뉴형 프로그램으로 수정했습니다.

사용자는 메뉴에서 신규 식재료 등록, 냉장고 식재료 전체 출력, 식비 점유율 및 남은 예산 확인, 지출 등급 판정, 특별 칭호 확인, 유통기한 임박 식재료 확인, 프로그램 종료 기능을 선택할 수 있습니다. 사용자가 0번 종료 메뉴를 선택하면 **break**를 통해 프로그램이 종료되도록 구현했습니다.

또한 기존 2차 과제에서 구현했던 입력, 계산, 출력, 판정 기능을 여러 개의 함수로 분리했습니다. `show_menu()`, `add_food()`, `show_food_list()`, `calculate_budget_rate()`, `show_budget_status()`, `judge_spending()`, `check_title()`, `check_expiring_food()` 함수를 정의하여 메인 로직에서는 함수 호출 중심으로 코드가 실행되도록 구성했습니다.

`calculate_budget_rate(total, budget)` 함수는 매개변수를 전달받아 식비 점유율을 계산하고, `return`을 통해 결과값을 반환하도록 구현했습니다. `add_food()` 함수에서는 프로그램 전체에서 사용하는 `total_price` 값을 수정하기 위해 `global` 키워드를 사용했습니다.

기술회고에는 이 포인트 넣으면 좋음

Troubleshooting & 기술 회고

문제 1: 기존 코드가 한 번 실행된 뒤 바로 종료되는 문제

원인: 2차 과제 코드에서는 식재료 3개를 입력받고 결과를 출력한 뒤 프로그램이 끝나는 구조였기 때문에 사용자가 추가 기능을 선택할 수 없었습니다.

해결: `while True`를 사용하여 메인 메뉴가 반복적으로 출력되도록 수정했습니다. 사용자가 0번 종료 메뉴를 선택했을 때만 `break`로 반복문을 종료하도록 구현했습니다.

문제 2: 함수로 나누면서 `total_price` 값이 제대로 누적되지 않는 문제

원인: `total_price`는 프로그램 전체에서 사용하는 전역변수인데, `add_food()` 함수 내부에서 값을 변경하려고 하자 지역변수처럼 처리되어 오류가 발생할 수 있었습니다.

해결: `add_food()` 함수 내부에 `global total_price`를 작성하여 함수 안에서도 전역변수 `total_price` 값을 수정할 수 있도록 했습니다.

문제 3: 등록된 식재료가 없을 때 목록 출력이 어색한 문제

원인: 식재료 리스트가 비어 있는 상태에서 전체 목록 출력이나 유통기한 확인 기능을 실행하면 출력할 데이터가 없었습니다.

해결: `len(food_names) == 0` 조건을 사용하여 등록된 식재료가 없을 경우 안내 문구를 출력하도록 했습니다.

문제 4: 같은 계산식이 여러 메뉴에서 반복되는 문제

원인: 식비 점유율 계산식이 예산 확인, 지출 등급 판정, 특별 칭호 확인 메뉴에서 반복적으로 사용되었습니다.

해결: `calculate_budget_rate(total, budget)` 함수를 따로 만들어 계산식을 한 곳에서 처리하도록 수정했습니다. 이 함수는 매개변수로 총 지출 금액과 예산을 전달받고, 계산된 식비 점유율을 `return`으로 반환합니다.

결론적으로, 네 3차 과제는 ***메뉴형 + 함수형으로 리팩토링***하는 게 핵심이야. 아까보다 조건이 뻣세졌으니, 최종 코드는 반드시 함수 3개 이상 / 매개변수 / `return` / `global` / `while True` / `break`가 눈에 보이게 들어가야 해.

